

SchemaForge v2.0

Implementation Plan

Autonomous Database Migration Reliability Agent

From the current repository to a judge-ready, evidence-backed migration agent with a hard production approval boundary.

Document	Engineering implementation plan
Baseline	Repository state as of 26 September 2026 and <code>SchemaForge_v2_Engineering_Playbook.pdf</code>
Stack	TrueForge, MCP (TypeScript, <code>@modelcontextprotocol/sdk</code>), PostgreSQL 16, Docker Compose
Scope	PostgreSQL only; one e-commerce schema; one failing and one safe scenario
Estimated effort	About 7.5 focused engineering hours across 6 phases

Guiding principle

The agent may reason and rehearse autonomously. Production mutation is a separate, action-bound, human-authorised capability that the agent process cannot forge, reuse, or apply to a schema that has drifted.

Contents

1	Objective and Definition of Done	2
2	Current State Assessment	2
2.1	Component inventory	2
2.2	Findings	3
3	Target Architecture	5
3.1	Trust boundaries	5
4	Implementation Phases	5
4.1	Phase 0: Unblock the pipeline (1.5 h)	5
4.2	Phase 1: Safety core (2 h)	6
4.3	Phase 2: Deterministic rehearsal and evidence (1.5 h)	8
4.4	Phase 3: Agent integration (1 h)	9
4.5	Phase 4: Automated tests (1 h)	9
4.6	Phase 5: Demo hardening (0.5 h)	10
5	Schedule	10
6	Five-Minute Demo Run Sheet	11
7	Risk Register	11
8	Out of Scope	12
A	File Change Map	12

1 Objective and Definition of Done

SchemaForge turns a natural-language migration request into an evidence-backed decision. The repository already contains a skeleton of every component described in the playbook. This plan closes the gap between that skeleton and a system whose safety claims are enforced by code and by the database, not only by documentation.

Definition of done (from the playbook, made testable)

1. A TrueForge session inspects the real PostgreSQL target through the read-only role.
2. Request A (“*make `users.email` NOT NULL*”) is rehearsed on an isolated copy of representative data, the 14 seeded NULL emails are found, and the deterministic decision engine returns **DO NOT APPLY**. Production is untouched.
3. Request B (“*add a nullable `archived_at` column to users*”) passes all checks, TrueForge freezes, a human mints a signed approval bound to that exact SQL and schema fingerprint, and the separate executor applies it.
4. Read-only post-apply verification confirms the new column and unchanged invariants.
5. Replaying the same approval, editing the SQL, or changing the schema before apply is rejected, and each rejection is covered by an automated test.

2 Current State Assessment

2.1 Component inventory

The MCP server type-checks cleanly (`tsc --noEmit` reports no errors). The table records whether each playbook capability works end to end.

Capability	Location	State	Note
Two PostgreSQL 16 containers	<code>docker-compose.yml</code>	✓	prod :5433, shadow :5434
Read-only role <code>sf_reader</code>	<code>docker/init-prod.sql</code>	✓	SELECT grants and default privileges
Schema inspection and fingerprint	<code>mcp-server/src/tools/inspect_schema.ts</code>	✓	row counts are <code>reltuples</code> estimates
Read-only query tool	<code>mcp-server/src/tools/readonly_query.ts</code>	~	regex and row cap; no read-only transaction
Dependency analysis	<code>mcp-server/src/tools/analyze_dependencies.ts</code>	~	database objects only, no repository scan
Shadow rehearsal	<code>mcp-server/src/tools/rehearse_migration.ts</code>	~	not isolated; transaction bug (F2)
Gated executor	<code>mcp-server/src/tools/execute_migration.ts</code>	~	unsigned token, no drift check
Post-apply verification	<code>mcp-server/src/tools/verify_production.ts</code>	✓	table name interpolated (F12)
Seeded edge cases	<code>shadow/seed-edge-cases.sql</code>	✓	14 NULL emails, 3 duplicates, and more
Invariant and fingerprint library	<code>verification/*.ts</code>	~	not wired into the server

Capability	Location	State	Note
Execution-mode classifier	types.ts (MigrationClassification)	×	type exists, unused
Decision engine	types.ts (DecisionPacket)	×	type exists, unused
Approval minting and ledger	—	×	nothing issues or consumes tokens
Automated tests	—	×	none in repository

Legend: ✓ implemented ~ partial × absent

2.2 Findings

Each finding was confirmed by reading the referenced file. Severity reflects impact on the definition of done: **CRITICAL** blocks the demo or voids a safety claim, **HIGH** weakens a safety claim, **MEDIUM** and **LOW** affect quality or honesty of evidence.

ID	Severity	Finding	Evidence
F1	CRITICAL	Environment variable names disagree. The server reads SF_PROD_READONLY_URL, SF_PROD_WRITE_URL and SF_SHADOW_URL; .env.example and agent.yaml provide PROD_READONLY_URL, PROD_DATABASE_URL and SHADOW_DATABASE_URL. No pool is created, so every tool call fails.	mcp-server/src/db.ts lines 38–71; trueforge-config/agent.yaml
F2	CRITICAL	Transactions span different connections. db.query() checks out a fresh pooled client per call, so BEGIN, the migration and COMMIT run in separate sessions. ROLLBACK does nothing, and a failed multi-statement migration can be half applied.	db.ts query(); rehearse_migration.ts; execute_migration.ts
F3	CRITICAL	Demo data is not where the demo needs it. Production is created with email NOT NULL and zero rows, and the shadow init mirrors that strict schema. The nullable schema and 14 NULL emails exist only in shadow/seed-*.sql, which Compose never loads. Request A is already satisfied in production and rehearses with no violations.	docker/init-prod.sql, docker/init-shadow.sql, shadow/seed-schema.sql
F4	HIGH	The reset script authenticates as postgres/postgres; the shadow container's superuser is sf_shadow.	shadow/reset-shadow.sh

ID	Severity	Finding	Evidence
F5	CRITICAL	Approval tokens are unsigned and supplied by the caller, including the <code>used</code> flag. The agent can fabricate a valid token for any SQL, and single use is not enforced server-side.	<code>execute_migration.ts</code> checks 1–4; <code>ApprovalToken</code> in <code>types.ts</code>
F6	CRITICAL	The pre-apply schema drift check is described in the header comment but not implemented, and the token carries no fingerprint.	<code>execute_migration.ts</code> lines 14–17
F7	HIGH	No statement classifier. Tier-3 prohibitions exist only in YAML, and every migration is wrapped in <code>BEGIN</code> , so <code>CREATE INDEX CONCURRENTLY</code> fails.	<code>MigrationClassification</code> unused
F8	HIGH	The production write pool lives in the same process as the agent’s read and rehearsal tools, which contradicts the playbook’s credential rule.	<code>prodWrite</code> pool in <code>db.ts</code>
F9	HIGH	Rehearsal commits into one long-lived shadow database, so each run mutates the baseline for the next. Evidence is not deterministic.	<code>rehearse_migration.ts</code>
F10	MEDIUM	<code>verification/</code> is a separate package without installed dependencies and is not imported; fingerprinting is implemented twice.	<code>verification/package.json</code> ; <code>inspect_schema.ts</code>
F11	MEDIUM	No deterministic decision engine; <code>APPLY</code> , <code>REVIEW</code> or <code>DO NOT APPLY</code> is left to the LLM.	<code>DecisionPacket</code> unused
F12	MEDIUM	<code>table_name</code> is interpolated directly into SQL.	smoke test in <code>verify_production.ts</code>
F13	MEDIUM	The read-only tool relies on regex plus role grants; sessions are not forced read-only.	<code>readonly_query.ts</code>
F14	LOW	Row counts come from <code>pg_class.reltuples</code> and must be flagged as estimates.	<code>inspect_schema.ts</code> line 80
F15	LOW	The README describes <code>src/server.ts</code> , HTTP port 3100 and a root <code>npm install</code> ; the real server is <code>mcp-server/</code> on stdio.	<code>README.md</code>
F16	MEDIUM	No automated tests for any safety property.	repository

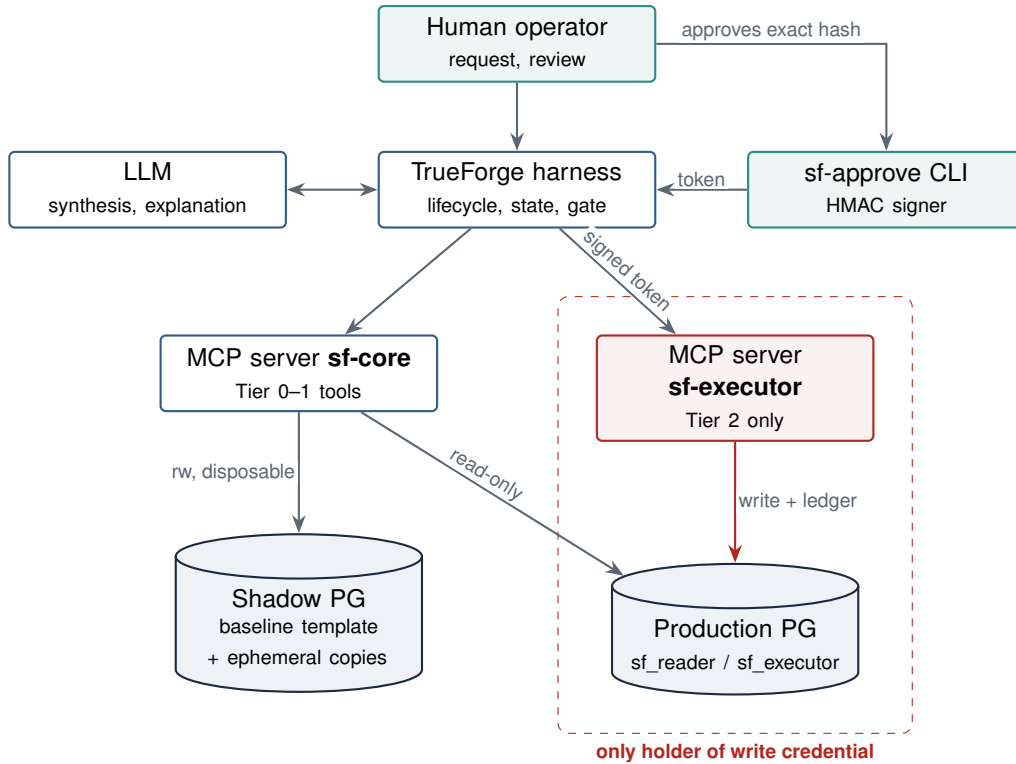
Honest evidence note

The playbook’s sample packet quotes 182,340 rows inspected. The seed currently creates 500 users. Either scale the seed (Phase 2) or show the real number. The packet must never display

a figure that was not measured.

3 Target Architecture

The main change is splitting the MCP layer into two processes with different credentials, and moving approval minting to a human-side tool whose signing secret the agent never sees.



3.1 Trust boundaries

- **sf-core** holds `SF_PROD_READONLY_URL` and `SF_SHADOW_URL` only. Its tools are `db_inspect_schema`, `db_run_readonly_query`, `analyze_dependencies`, `rehearse_migration`, `build_decision_packet` and `verify_production`.
- **sf-executor** holds `SF_PROD_WRITE_URL` and `SF_APPROVAL_SECRET` and exposes one tool, `execute_migration`.
- **sf-approve** runs in the human's terminal, displays the decision packet, requires typing the migration hash prefix, and signs. The default answer is reject.
- Production write access uses a dedicated `sf_executor` role (DDL on `public`, INSERT on the ledger), not the container superuser `sf_admin`.

4 Implementation Phases

Each phase lists tasks, touched files and acceptance criteria. Phases 0 and 1 are mandatory before any demo rehearsal, and everything later builds on them.

4.1 Phase 0: Unblock the pipeline (1.5 h)

0.1 Single validated config (F1). Add `mcp-server/src/config.ts`, which parses `process.env` with `zod`, uses one naming scheme (`SF_*`), and fails fast with a clear message. Update

.env.example and agent.yaml to match.

0.2 Connection-scoped transactions (F2). Add withClient and withTransaction to DatabaseManager, and remove every bare BEGIN/COMMIT issued through db.query(). Use SET LOCAL for timeouts.

0.3 Align demo data (F3). Production represents the real target, so it must hold the nullable schema and representative rows. Load shadow/seed-schema.sql, seed-data.sql and seed-edge-cases.sql into prod through Compose init scripts (prefixes 02-, 03-, 04-), then apply role grants. Shadow is populated from prod (Phase 2), not from its own schema file.

0.4 Fix the reset script (F4). Default to sf_shadow credentials read from .env.

0.5 README truth pass (F15). Correct paths, transport and install steps.

```
// db.ts -- every multi-statement unit of work runs on ONE session
async withTransaction<T>({
  pool: PoolName,
  fn: (client: pg.PoolClient) => Promise<T>,
  opts: { timeoutMs?: number; readOnly?: boolean } = {},
}): Promise<T> {
  const client = await this.getPool(pool).connect();
  try {
    await client.query(opts.readOnly ? 'BEGIN READ ONLY' : 'BEGIN');
    const ms = Math.trunc(opts.timeoutMs ?? this.statementTimeoutMs);
    await client.query(`SET LOCAL statement_timeout = ${ms}`);
    const result = await fn(client);
    await client.query('COMMIT');
    return result;
  } catch (err) {
    await client.query('ROLLBACK').catch(() => {});
    throw err;
  } finally {
    client.release();
  }
}
```

Acceptance criteria

- docker compose down -v && docker compose up -d yields prod with 500 or more users and exactly 14 NULL emails.
- The server starts with the .env.example values and all tools connect.
- The test migration ALTER TABLE users ADD COLUMN x int; SELECT 1/0; leaves no column x behind.

4.2 Phase 1: Safety core (2 h)

1.1 Statement classifier (F7)

Create mcp-server/src/safety/classifier.ts. Parse with a real SQL parser (pgsql-ast-parser, exact pinned version) rather than regex, split multi-statement input, and classify each statement. **Fail closed:** anything unparseable is PROHIBITED for execution and REVIEW for the decision.

Statement pattern	Class	Execution mode
ADD COLUMN (nullable, no volatile default)	TRANSACTIONAL	single transaction with lock_timeout
SET NOT NULL, ADD CONSTRAINT	TRANSACTIONAL	validate first (see Phase 2)

CREATE/DROP INDEX CONCURRENTLY,	NON_	autocommit, one statement
REINDEX CONCURRENTLY	TRANSACTIONAL	per call
DROP TABLE, TRUNCATE, DROP	PROHIBITED	rejected at every layer
DATABASE, DROP SCHEMA		
GRANT, REVOKE, ALTER ROLE, CREATE	PROHIBITED	rejected at every layer
ROLE		
DELETE/UPDATE without WHERE, DROP	PROHIBITED	escalated, not executable
COLUMN		
Mixed transactional and	PROHIBITED	must be split into separate
non-transactional		migrations

The classifier runs in `rehearse_migration` and `build_decision_packet`, and again inside `execute_migration`. The `sf_executor` role has no TRUNCATE, no role management, and does not own tables it must not drop, so the database remains a boundary even if the classifier is wrong.

1.2 Signed, action-bound approvals (F5, F6)

```
// safety/approval.ts -- shared by sf-approve (sign) and sf-executor (verify)
export interface ApprovalPayload {
  migration_hash: string; // sha256 of normalised SQL
  target: 'prod';
  schema_fingerprint: string; // fingerprint captured at rehearsal time
  action: string; // human-readable summary shown at approval
  nonce: string; // crypto.randomUUID()
  issued_at: string;
  expires_at: string; // issued_at + SF_APPROVAL_TTL_SECONDS
}

export interface SignedApproval { payload: ApprovalPayload; sig: string }

export function sign(p: ApprovalPayload, secret: Buffer): SignedApproval {
  const sig = createHmac('sha256', secret).update(canonicalJson(p)).digest('hex');
  return { payload: p, sig };
}

export function verify(a: SignedApproval, secret: Buffer): boolean {
  const expected = createHmac('sha256', secret)
    .update(canonicalJson(a.payload)).digest();
  const given = Buffer.from(a.sig, 'hex');
  return given.length === expected.length && timingSafeEqual(given, expected);
}
```

The `used` field is removed from the token. Single use is enforced by a ledger in production, written in the same transaction as the migration:

```
CREATE SCHEMA sf_audit;
CREATE TABLE sf_audit.consumed_approvals (
  nonce          uuid          PRIMARY KEY,
  migration_hash text          NOT NULL,
  fingerprint    text          NOT NULL,
  consumed_at    timestampz    NOT NULL DEFAULT now(),
  outcome        text          NOT NULL
);
```

1.3 Executor sequence

1. Verify the HMAC signature, expiry and `target`.
2. Recompute `sha256` of the normalised SQL; it must equal `migration_hash`.

3. Re-classify and reject anything PROHIBITED.
4. Recapture the schema fingerprint through `sf_reader`. A mismatch returns `SCHEMA_DRIFT` and requires a new rehearsal.
5. *Transactional*: `BEGIN; SET LOCAL lock_timeout = '3s';` insert the ledger row (the primary key blocks replay); run the SQL; `COMMIT`.
6. *Non-transactional*: commit the ledger row first with outcome `started`, run the statement in `autocommit`, then update the outcome.
7. Return the measured duration labelled `is_estimate: false`.

1.4 Process split (F8)

Move `execute_migration` into `mcp-server/src/executor.ts`, a second entry point started with `npm run start:executor`. Register both servers in `agent.yaml`; the `sf-core` environment omits the write URL and the approval secret. Add a startup assertion in `sf-core` that refuses to boot if `SF_PROD_WRITE_URL` is present.

Acceptance criteria

- A forged token (wrong secret), edited SQL, expired token, replayed nonce, drifted schema and a `DROP TABLE` each produce a distinct rejection code.
- `CREATE INDEX CONCURRENTLY` executes successfully outside a transaction.
- `grep -r SF_PROD_WRITE_URL` matches only executor code and config.

4.3 Phase 2: Deterministic rehearsal and evidence (1.5 h)

2.1 Realistic, resettable shadow (F9). Build `sf_baseline` in the shadow container from a `pg_dump` of prod (schema and data). Each rehearsal runs `CREATE DATABASE sf_rehearsal_<id> TEMPLATE sf_baseline`, executes there, and drops it afterwards. Runs are isolated and repeatable, and support non-transactional DDL. Add `shadow/refresh-baseline.sh`.

2.2 Scale the dataset. Parameterise the `generate_series` bounds in `seed-data.sql` (for example 180k users and 400k orders) while keeping the edge-case rows deterministic through `setseed(0.42)`.

2.3 One verification library (F10). Move `verification/*.ts` into `mcp-server/src/verification/`, delete the duplicate fingerprint code in `inspect_schema.ts`, and call `runAllChecks` before and after the forward SQL.

2.4 Safe-pattern synthesis hints. Guide the model through the system prompt toward low-lock patterns. For `NOT NULL` on a large table:

```
ALTER TABLE users ADD CONSTRAINT users_email_nn CHECK (email IS NOT NULL) NOT VALID;
ALTER TABLE users VALIDATE CONSTRAINT users_email_nn; -- SHARE UPDATE EXCLUSIVE lock
ALTER TABLE users ALTER COLUMN email SET NOT NULL; -- PG12+: skips the full scan
ALTER TABLE users DROP CONSTRAINT users_email_nn;
```

2.5 Deterministic decision engine (F11). A new tool, `build_decision_packet`, fills `DecisionPacket` from rehearsal output using fixed rules. The LLM explains the verdict but cannot change it.

2.6 Evidence labelling (F14). Use an exact `count(*)` for the target table; estimates keep `is_estimate: true` and render with a $\approx$ prefix.

2.7 Rollback rehearsal. For reversible classes, run the rollback in the same ephemeral database and compare its fingerprint with the baseline.

Rule (first match wins)	Decision	Rationale
Any statement PROHIBITED or unparseable	DO NOT APPLY	Tier 3
Forward rehearsal failed, or violations > 0	DO NOT APPLY	data incompatible
Post-migration invariant check failed	DO NOT APPLY	integrity loss
Schema drift between inspection and rehearsal	REVIEW	stale evidence
Rollback required but not verified	REVIEW	recovery unproven
Application references flagged > 0	REVIEW	compatibility risk
Otherwise	APPLY	all evidence green

Acceptance criteria

- Request A rehearsed three times in a row yields identical packets (ignoring timestamps and durations) with `violations_found = 14`.
- Shadow `sf_baseline` is unchanged after any rehearsal.
- Every numeric field in the packet is tagged as measured or estimated.

4.4 Phase 3: Agent integration (1 h)

- `trueforge-config/agent.yaml`: two MCP servers, updated tool tiers (`build_decision_packet` is Tier 0), and a gate configured to pause before any `execute_migration` call and resume with the signed token as input.
- `trueforge-config/system-prompt.md`: a fixed tool order (inspect, dependencies, synthesise, rehearse, packet, gate), an instruction never to call the executor without a token supplied by the harness, and never to restate estimates as measurements.
- Prompt-injection hardening: database comments and row data returned by tools are treated as data. The human approves the packet, not model prose.
- `sf-approve` CLI (`mcp-server/src/cli/approve.ts`): reads the packet, refuses to sign unless the decision is APPLY, and requires typed confirmation of the first 8 hash characters.
- Repository reference scan (optional, time-boxed to 20 minutes): `analyze_dependencies` searches a configured `SF_APP_REPO_PATH` for table and column names.

Acceptance criteria

Both scenarios run end to end in a live TrueForge session without manual SQL, and the session visibly freezes at the gate until the `sf-approve` output is provided.

4.5 Phase 4: Automated tests (1 h)

Use `vitest` (pinned) in `mcp-server`. Unit tests need no database; integration tests run against the Compose databases and reset them in `beforeAll`.

ID	Test	Type	Covers
T1	Every row of the classifier table maps to the expected class	unit	F7
T2	Unparseable input and mixed multi-statement input fail closed	unit	F7
T3	HMAC sign and verify; tampered payload, wrong key and truncated signature rejected	unit	F5
T4	Decision engine rule table, one case per row	unit	F11
T5	A failing mid-migration statement leaves no partial change	integration	F2
T6	Request A: 14 violations, DO NOT APPLY, prod fingerprint unchanged	integration	F3, F9
T7	Request B: approve, execute, verify the column exists	integration	definition of done
T8	Replaying the T7 token is rejected by the ledger	integration	F5
T9	Schema altered after approval returns SCHEMA_DRIFT	integration	F6
T10	<code>sf_reader</code> cannot write even if the regex is bypassed	integration	F13
T11	<code>sf-core</code> refuses to start when a write URL is present	unit	F8
T12	A malicious <code>table_name</code> in <code>verify_production</code> is rejected	unit	F12

Acceptance criteria

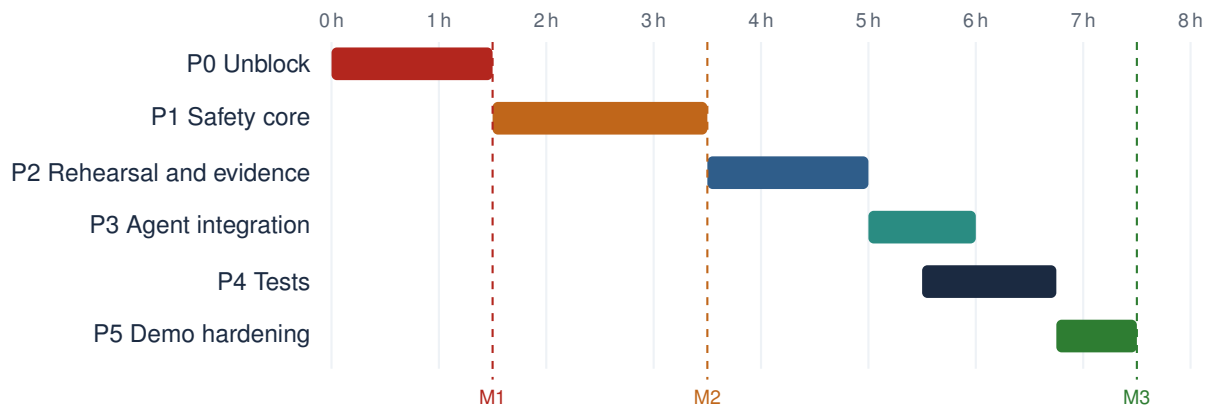
`npm test` passes locally after a clean `docker compose down -v`, and every **CRITICAL** finding has at least one test.

4.6 Phase 5: Demo hardening (0.5 h)

- `scripts/demo-reset.sh`: recreate containers, refresh the baseline, verify counts.
- Two full dry runs against the five-minute run sheet (Section 6).
- Record a backup video of a successful run.
- Feature freeze: once Phase 5 starts, only bug fixes.

5 Schedule

Estimates assume a single engineer on the critical path. Two people can overlap Phases 2 and 3 once Phase 1 lands.



Milestone	Name	Exit condition
M1	Pipeline runs	Phase 0 acceptance met; every tool call succeeds against real databases
M2	Gate enforced	All five token and drift rejection paths are demonstrable
M3	Demo ready	Two clean dry runs, tests green, backup video recorded

6 Five-Minute Demo Run Sheet

Time	Beat	What must be real on screen
0:00–0:45	Problem and architecture	Section 3 diagram; point at the red executor boundary
0:45–2:00	Request A	Live <code>db_inspect_schema</code> output showing nullable <code>email</code> in prod
2:00–3:00	Rehearsal fails	Ephemeral database created; 14 violations with sample ids; packet says DO NOT APPLY; prod fingerprint unchanged
3:00–3:40	Request B	Additive nullable <code>archived_at</code> ; all checks green
3:40–4:20	Gate	TrueForge frozen; <code>sf-approve</code> shows the hash and defaults to reject; human types the hash prefix
4:20–5:00	Execute and verify	Executor applies; <code>verify_production</code> confirms; optionally replay the token live to show the rejection

7 Risk Register

Risk	Impact	Mitigation
TrueForge gate semantics differ from the assumptions in <code>agent.yaml</code>	CRITICAL	Validate gate pause and resume in the first hour of Phase 3. Fallback: the executor itself blocks without a token, and the gate only displays the packet
SQL parser rejects valid PostgreSQL syntax	HIGH	Fail closed to REVIEW; keep demo SQL within parser coverage; unit test both demo migrations
Template database creation fails while <code>sf_baseline</code> has active connections	MEDIUM	Never connect tools to the baseline; the refresh script terminates backends first

Risk	Impact	Mitigation
LLM synthesises different SQL across runs	MEDIUM	Temperature is already 0.1; the decision depends on rehearsal evidence, not SQL text; the packet shows the hash
Secrets committed from <code>.env</code>	HIGH	Confirm <code>.env</code> is git-ignored before the first commit; generate the approval secret locally per machine
A larger seed slows container start	LOW	Keep scale behind a variable; size the demo default to start in under 30 seconds

8 Out of Scope

Following the playbook's scope cuts, these stay out: multi-database support, Kubernetes, Slack or Jira integration, user authentication, any web UI beyond terminal output, generic multi-agent orchestration, and automatic rollback of irreversible changes.

A File Change Map

File	Phase	Change
<code>mcp-server/src/config.ts</code>	0	new: zod-validated environment
<code>mcp-server/src/db.ts</code>	0	<code>withClient</code> , <code>withTransaction</code> ; config-driven pools
<code>docker-compose.yml</code> , <code>docker/init-*.sql</code>	0	prod loads seed files; <code>sf_executor</code> role; ledger schema
<code>shadow/reset-shadow.sh</code>	0	correct credentials
<code>.env.example</code> , <code>README.md</code>	0	align names, paths and transport
<code>mcp-server/src/safety/classifier.ts</code>	1	new
<code>mcp-server/src/safety/approval.ts</code>	1	new: sign, verify, canonical JSON
<code>mcp-server/src/executor.ts</code>	1	new entry point hosting <code>execute_migration</code>
<code>mcp-server/src/tools/execute_migration.ts</code>	1	signature, ledger, drift check, execution modes
<code>mcp-server/src/index.ts</code>	1	remove executor tool; refuse a write URL
<code>shadow/refresh-baseline.sh</code>	2	new: dump prod into <code>sf_baseline</code>
<code>mcp-server/src/tools/rehearse_migration.ts</code>	2	ephemeral template database; invariants; rollback
<code>mcp-server/src/verification/*</code>	2	moved from <code>verification/</code>
<code>mcp-server/src/tools/build_decision_packet.ts</code>	2	new: rule engine
<code>mcp-server/src/tools/verify_production.ts</code>	2	identifier validation
<code>mcp-server/src/tools/readonly_query.ts</code>	2	BEGIN READ ONLY

File	Phase	Change
trueforge-config/agent.yaml, system-prompt.md	3	two servers, gate, tool order
mcp-server/src/cli/approve.ts	3	new: human-side signer
mcp-server/test/	4	new: T1-T12
scripts/demo-reset.sh	5	new